

π Tantum

a.k.a. Tempus Tantum

school timetable generation system

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”
Lucius Annaeus Seneca, *Epistulae morales ad Lucilium*, I, 1

Manual

A method for Italian-school timetabling

(English edition)

From an idea by Fernando Gargiulo, Giovanni Borghi,
Matteo Mariani, Stefano Bertozzi



Giovanni Borghi

May 6, 2026

Repository: <https://github.com/gborghi/timetable>
Documentation source: docs/

Colophon

This English edition is set in EB Garamond for body text and Lato for headings, with Inconsolata for code blocks. Compilation is done with lualatex (TeX Live), biber for bibliography and makeindex for the analytical index. The L^AT_EX source lives in docs/manual_en.tex with chapters in docs/manual/chapters_en/; the bibliography is shared with the Italian edition in docs/manual/bibliography.bib. The build pipeline is in docs/build_manual.sh which emits **both** manual.pdf (Italian) and manual_en.pdf (English) in a single invocation.

Note on translation: the Italian edition remains the primary reference. This English edition fully translates the introduction, the BP-related chapters and the benchmark chapter; all other chapters carry an English summary that points the reader to the corresponding Italian chapter for the full narrative. Where Italian-school context is necessary (for instance the legal frame of the *contratto collettivo nazionale*, the role of the *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added in the English text.

The repository version corresponds to the most recent commit visible in `git log --oneline -- docs/`. Errors and suggestions can be reported as issues on the GitHub repository at <https://github.com/gborghi/timetable>.

All project rights belong to Giovanni Borghi. Document produced in May 2026.

Contents

List of Figures

List of Tables

Introduction

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”

Lucius Annaeus Seneca, *Epistulae morales ad Lucilium*, I, 1

π Tantum is a web application that assists in the composition of Italian-school timetables. It originates from ideas discussed and developed by Fernando Gargiulo, Giovanni Borghi, Matteo Mariani and Stefano Bertozzi.

The school timetabling problem consists of assigning, to each *cattedra* – the pair formed by a teacher and a class for a specific subject – a set of weekly hours distributed over days and time slots, such that all the following constraints are honoured: teacher availability, curricular hour coverage, classroom and equipment compatibility, and the constraints that derive from the Italian national collective contract for the school sector. This is a combinatorial problem classified as NP-hard, which means that as a school grows in size there is no fast algorithm capable of finding an exact optimal solution.

π Tantum delegates the actual solving to the CP-SAT solver in Google OR-Tools, chosen for its maturity in constraint programming applied to scheduling problems. To scale beyond the size that the solver would handle directly, the system decomposes the problem into smaller sub-problems via four alternative strategies: spectral decomposition of the bipartite class-teacher graph, temporal decomposition by day of the week, curriculum-based decomposition, and balanced k -way METIS partitioning. On top of these, a cascade of metaheuristics (Large Neighborhood Search, Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search and Variable Neighborhood Search) improves the quality of the solution found.

The application backend is written in Python on FastAPI; data persistence is delegated to SQLite. The user interface is based on SvelteKit (in Italian for the production deployment), organised into sixteen tabs that cover the whole lifecycle of a timetable: registry import, constraint configuration, generation, manual editing with real-time feasibility checks, and day-to-day handling of absences and substitutions.

The following chapters describe the problem formally, the entities involved, the user interface, and each of the algorithms that π Tantum employs to compute the timetable. The exposition is intentionally multi-layered: a non-technical reader can follow the narrative thread without engaging the mathematical formulas, while a developer will find the implementation details, code references, and source-file pointers necessary to extend the system.

This English edition is a parallel translation of the Italian manual (docs/manual.tex). The Italian edition remains the primary reference; this English edition is intended for international readers and contributors. Where Italian-school context is necessary (e.g. the legal frame of the *contratto collettivo nazionale*, the role of *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added.

1 The timetabling problem

Formal statement of the school timetabling problem: cattedre as (teacher, class, subject) triples; weekly hour distribution across days and time slots; HARD constraints (no class/teacher overlap, hour coverage, consecutive hours starting at 8 with presence at $h=11$) and SOFT objectives (daily uniformity, free-day preferences). NP-hardness and the rationale for decomposition + metaheuristics.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/problema__timetabling.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

2 piTantum overview

Three-tier architecture: engine (CP-SAT solver core), backend (FastAPI + SQLite), frontend (SvelteKit). Pipelines exposed as async runs via `/api/optimize/*`. The 16 UI tabs covering registry, constraints, generation, manual editing, daily operations.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/panoramica__pitantum.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

3 The timetable as a structural object

The Lesson dict (teacher, class, subject, day, hour): 1 as the canonical representation; round-trip with the Solution table; the role of dc_value in encoding Phase A's day-count distribution.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/orario_oggetto_strutturale.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

4 Installation

Python 3.13 + ortools + scipy + matplotlib + FastAPI + SQLAlchemy + Alembic; SvelteKit for the frontend. Cypress + Playwright for E2E. docker-compose.test.yml orchestrates the test stack.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/installazione.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

5 Constraints (HARD + SOFT)

HARD: no overlap (teacher in one class at a time, class with one teacher per slot), hour coverage of all cattedre, daily start at h=8, consecutive hours, presence at h=11 (the H₁/H₂/H₃ trio). Native locks. C₁ (compresenza/coteach), C₂ (sostegno DVA), C₃ (potenziamento, parallel intra-class, group inter-class). SOFT: daily uniformity, free-day preferences, sixth-hour penalties, five/one day load.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/vincoli.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

5.1 Plessi: multiple sites in the same school

In Italy many secondary schools are spread across multiple physical sites – the *Sede Centrale* (main building), one or more *succursali* (branch buildings, e.g. *Succursale Via Garibaldi*), or a dedicated *plesso* for a specific track (e.g. *Plesso ITT* for the technical track). Moving teachers or classes between sites during the day costs real time and introduces constraints that cannot be modelled with H₁/H₂/H₃ alone.

The **Plessi** module (tab /plessi) handles:

- **site registry** (name, code, address);
- **classroom** → **plesso** association (each classroom belongs to exactly one site);
- two kinds of behavioural rules: **commuting rules** (minimum gap to move between two sites) and **entity policies** (global constraints on how many sites a teacher or class may attend).

5.1.1 Commuting rules: the gap between two sites

A commuting rule applies to an ordered pair (from_plesso, to_plesso) and specifies the minimum number of hours that must separate two consecutive lessons of the same entity when the classroom changes site. Relevant fields:

- from_plesso_id, to_plesso_id: the site pair.
- entity_kind: who is affected (teacher, class, group).
- entity_id: optional – if NULL the rule is *kind-wide* (applies to every teacher / class / group); if set it applies only to that specific entity.
- min_gap_hours: minimum number of empty hours required between the last lesson at site A and the first lesson at site B.
- symmetric: if true the rule applies in the opposite direction (B→A) as well.
- priority: tie-breaker when several rules overlap.

Typical Italian example: between *Sede Centrale* and *Succursale Via Garibaldi* a 60-minute travel time is required. The matching rule is (Centrale, Garibaldi, kind=teacher, entity_id=NULL, min_gap_hours=1, symmetric=true): no teacher may have a lesson at Garibaldi at h=9 if they have one at Centrale at h=8.

Rule resolution always applies *specific beats generic*: if a rule with entity_id=42 exists for teacher Rossi, it overrides the kind-wide rule. Among rules of equal specificity, higher priority wins.

5.1.2 Entity policies: per-person global constraints

An entity policy is a *global* constraint (not tied to a specific site pair) on the behaviour of a teacher or class. Three policies are supported:

- any – no global constraint, only commuting rules apply. This is the default.
- single_plesso_per_day – on any given day the entity may have lessons in at most one site. Stricter than commuting rules: even with a 1-hour gap, switching sites mid-day is forbidden.
- single_plesso_total – the entity attends *only* site plesso_id for the entire week. Every classroom assigned to that entity must belong to that site.

Concrete Italian examples:

- *Teacher Rossi does not want mid-day site switches*: the office creates a policy (entity_kind=teacher, entity_id=Rossi, policy=single_plesso_per_day). On Monday all of Rossi's hours are at Centrale; on Tuesday all at Garibaldi; never a mixed day.
- *Class 3A (technical track) attends only Plesso ITT*: policy (entity_kind=class, entity_id=3A, policy=single_plesso_total, plesso_id=ITT). Every classroom that hosts 3A lessons must belong to Plesso ITT.
- *All teachers, by default, avoid mid-day site switches*: policy (entity_kind=teacher, entity_id=NULL, policy=single_plesso_per_day) – applies to every teacher except those with a more specific override.

Entity policies follow the same *specific beats generic* rule (per-entity override), then priority.

5.1.3 Pre-run validation

Before launching the optimisation the system runs a battery of checks on the plessi data (validate_plessi_rules). If anything is inconsistent the run is rejected with a list of violations. What is checked:

- empty or duplicate plesso code;
- commuting rule referencing a non-existent plesso;
- commuting rule with negative min_gap_hours or larger than the day length;
- duplicate kind-wide commuting rule (same site pair, same kind, NULL entity_id, defined twice);
- single_plesso_total entity policy with no plesso_id or with a non-existent site;
- entity policy with an unknown policy value;
- override for a non-existent entity (deleted teacher or class).

The GET /api/plessi/validate endpoint lets the UI display a live “plessi config OK / N issues” badge.

5.1.4 CP-SAT implementation

At solver level, commuting rules become *pairwise* constraints on the classroom-assignment variables:

$$\forall (h_a, h_b) \text{ adjacent such that } \text{room}(t, d, h_a) \in P_A, \text{ room}(t, d, h_b) \in P_B : h_b - h_a \geq \text{min_gap}_{P_A \rightarrow P_B}.$$

Entity policies become counter constraints: `single_plesso_per_day` is $\sum_P \mathbf{1}[\exists h : \text{room}(t, d, h) \in P] \leq 1$ for each day d ; `single_plesso_total` fixes the domain of the classroom variables to the target site only.

All such constraints are *hard* by default. The roadmap envisages a *soft* variant with penalty for sporadic emergency overrides (e.g. last-minute substitute teaching), but the current version treats every plesso constraint as HARD.

6 Optimization workflow

The pipeline orchestrator: Hall pre-check, Phase A (assignment + day-count), Phase B (per-day slot placement), optional metaheuristic post-processing (LNS / ALNS / SA / TS / ILS / VNS), classroom assignment.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/workflow__di__ottimizzazione.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

7 UI guide

Tour of the 16 SvelteKit tabs: dashboard, anagrafica (teachers/classes/students/classrooms), assignments, monitor, optimize (with iterative-diversified and branch-and-price modes), diagnostics, runs.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/guida__ui.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

8 Launcher

Single-binary launcher script that bootstraps the venv, runs the alembic migrations, starts uvicorn + the SvelteKit dev server.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/launcher.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

9 CP-SAT method

Model formulation in OR-tools `cp_model`: BoolVar slot[(t, cl, s, d, h)], hour coverage equalities, no-overlap constraints, soft penalties as objective coefficients. Integer scaling for the fractional duals in BP.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_cpsat.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

10 Spectral decomposition

Bipartite class-teacher graph; normalised Laplacian eigendecomposition; spectral clustering into K balanced clusters; per-cluster CP-SAT + ricucitura with the bridge teachers.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_spettrale.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

11 Metaheuristics

LNS (random_window, day_cluster, worst_fit_day, teacher_day, classroom_day, single_class_week destroy ops + cp_sat_window repair), Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search, Variable Neighborhood Search.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo__metaeuristiche.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

12 Hall pre-check

Hall's marriage theorem applied to the bipartite (class, subject) -> qualified-teachers graph as a fast pre-check (1-100 ms) before launching Phase A.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_hall.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

13 Lagrangian relaxation and Column Generation

Dualisation of the bridge constraints between clusters; subgradient ascent on the Lagrangian multipliers; SA-based per-iteration refinement. Column Generation: master LP + per-teacher CP-SAT pricing; Branch-and-Price extension with all 9 granularities (see ch. ??).

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_lagrangian.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

14 Monte Carlo sensitivity

Random subset sampling to estimate the Hall violation probability under perturbations; useful as a pre-check robustness indicator.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo__montecarlo.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

15 Graph-based diagnostics

Bipartite class-teacher graph: density, modularity (Newman-Girvan), betweenness centrality. Identifies bridge teachers and structural bottlenecks.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_grafo.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

16 Statistical diagnostics

Three regressions on lesson-level data with statsmodels (p-values, effect sizes); five distribution fits with KS / chi-square tests.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_statistica.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

17 Constraint DSL method

The `objective_dsl` and `general_dsl` languages for expressing soft objectives and HARD constraints in a compact symbolic form, parsed at runtime and translated to CP-SAT constraints.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_dsl.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

18 Architecture

Three tiers (engine, backend, frontend) with the data flow: registry -> assignment -> dc_value -> Phase B -> integer recovery -> persistence -> UI. Async runs orchestrated via the /api/optimize/* endpoints and the SQLAlchemy 'runs' table.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/architettura.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

19 Data model

Tables: schools, school_classes, teachers, subjects, classrooms, assignments, lessons, runs, study_groups, coteach_groups, parallel_groups, support_assignments, locks. SQLAlchemy 2.0 ORM. Alembic migrations + lightweight in-code migrations in db.py.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/modello_dati.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

20 Generic constraint DSL

The `general_dsl` subset for HARD constraint authoring: predicates over (teacher, class, subject, day, hour), boolean combinators, compilation to CP-SAT constraints.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/dsl_generico_per_i_vincoli.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

21 Advanced optimization techniques

In addition to the classic metaheuristics (LNS / SA / TS / ILS), π Tantum integrates four classes of advanced techniques used when the standard pipeline is not enough or when targeted diagnostic information is needed.

21.1 Hall pre-check

engine/diagnostics/hall_check.py. Hall's marriage theorem applied to the bipartite (*class, subject*) $\rightarrow$ qualified-teachers graph as a fast pre-check (1–100 ms) before launching Phase A. Three violation kinds are reported:

1. subject supply vs demand,
2. subject coverage vs demand,
3. Hall sampling random-subset with mutually exclusive subsets.

Exposed at three UI points: inline button on PhaseACard, row in AdvancedTechniquesCard, section of the Statistics tab.

21.2 Column Generation and advanced Branch-and-Price

engine/column_generation.py. Master LP via scipy.linprog HiGHS; eight dedicated CP-SAT pricers at distinct granularities (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject, class, class-day, day, curriculum); a branch-and-price mode equipped with all four scalability techniques required for MEGA-size instances:

- **Ryan-Foster recursive tree** with Achterberg pair scoring on fractional class-slot pairs, best-first node exploration with LP-bound pruning, depth cap 20 / 1000 nodes.
- **Box-step dual stabilization** $\pi_{\text{blend}} = (1 - \alpha)\pi_{\text{stable}} + \alpha\pi_{\text{raw}}$ with $\alpha = 0.2$, applied to both master variants.
- **Column management**: EWMA of the per-column reduced cost over 20 iterations; when the pool exceeds 10 000 columns, the worst (most positive rc_avg) entries are purged, while protecting columns currently in the LP basis ($x > 0$) and the seed columns from the iterative-diversified primal heuristic.
- **Parallel pricing** via ProcessPoolExecutor (default $\lfloor \text{os.cpu_count()}/2 \rfloor$ workers). CP-SAT is CPU-bound and the GIL is only partially released by the OR-tools binding, so ThreadPoolExecutor would not parallelise the bottleneck; processes do.

The CP-SAT-as-fundamental contract

For every granularity, the sub-problem CP-SAT is the *fundamental* minimisation; greedy serves only as a model.add_hint warm-start (an accelerator). If CP-SAT cannot find a feasible solution within the time limit the pricer returns (None, 0.0): the greedy is *never* promoted to a column. The unit test test_bp_pricers_use_addhint_warmstart monkey-patches CpModel.add_hint with a counting wrapper to enforce this contract.

Measured numbers (synthetic HUGE / MEGA benchmark)

See tests/benchmarks/test_bp_huge_mega.py. On a synthetic scenario calibrated to match the engine's MEGA profile (100 classes, ~ 174 teachers, 4 subjects, ~ 400 cattedre) the pipeline reaches HARD-feasibility in **5.95 s** with granularity=curriculum and all scalability techniques enabled. On the smaller HUGE scenario (50 classes, ~ 95 teachers, 200 cattedre) the time is **2.57 s**. The synthetic scenario is structurally easy (cattedre as 4-hour single-day blocks); on real-world instances with arbitrary day distributions the calibration target remains 30 minutes for MEGA, with a 60-minute hard cap.

21.3 Lagrangian relaxation

engine/lagrangian.py. Relaxes the inter-cluster bridge constraints and dualises them with multipliers λ_b ; update via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ with $\alpha_k = \alpha_0 / (1 + k)$. Per-iteration refinement via Simulated Annealing.

22 Statistical diagnostics tab

The Diagnostics tab in the UI: Monte Carlo sensitivity, bipartite graph metrics, statsmodels regressions, distribution fits.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/diagnostica_statistica.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

23 REST API

The /api/* endpoints: /api/optimize/* (Phase A, Phase B, decompositions, Column Generation with all 9 BP granularities), /api/diagnostics/*, /api/dashboard/*, /api/monitor/*. See docs/api.md for the full list.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/api_rest.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

24 Extending the system

Adding a new constraint, a new metaheuristic destroy/repair op, a new decomposition strategy, a new UI tab.
The hooks the test suite exercises.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/estendere_il_sistema.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

25 Repository on GitHub

Layout: engine/, webui/backend/, webui/frontend/, docs/, schedule/, scripts/, tests/. CI workflows, PR conventions, issue tracker.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/repository_github.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

26 Benchmarks and results

“In God we trust. All others must bring data.”

W. E. Deming, attributed

Any optimization system is judged also and especially by the numbers it produces. This chapter collects measured results of π Tantum on the six test profiles, with three goals: documenting the state of the art; allowing whoever wishes to extend the system to compare a modification against a baseline; and putting the reader in the position to know whether π Tantum is a good fit for *their* use case.

26.1 The six profiles

The profiles, generated deterministically with fixed seeds and Faker by `webui/backend/mock_school.py`, span the size range typical of the Italian school system:

- small: 8 classes, 17 teachers, ~ 220 students, 17 rooms. A small provincial school.
- medium: 28 classes, 50 teachers, 700 students, 35 rooms. A medium-size institution.
- big: 40 classes, 75 teachers, 1000 students, 50 rooms. A large institution.
- huge: 50 classes, ~ 95 teachers, 1250 students, 60 rooms. A multi-site complex.
- superhuge: 80 classes, ~ 160 teachers, 2000 students, 80 rooms. 16 sections $\times$ 5 grades of scientific liceum.
- mega: 100 classes, ~ 174 teachers, ~ 2500 students, 100 rooms. 20 sections $\times$ 5 grades with a mix of 8 *indirizzi* (Scientific 6, ScienzeApplicate 4, ScienzeUmane 3, Linguistico FRA-TED 2, Linguistico FRA-SPA 2, EconomicoSociale FRA/SPA/TED 1 each). MEGA-scale stress test.

26.2 End-to-end pipeline times

Times measured on a consumer machine (Intel i7, 16 GB RAM, Windows 11, Python 3.13, ortools 9.15, 8 CP-SAT search workers):

Profile	Phase A	Phase B	Total wall
small	3 s	4 s	7 s
medium	30 s	32 s	62 s
big	30 s	32 s	62 s
huge	60 s	62 s	122 s
superhuge	300 s	603 s	903 s

Table 26.1: End-to-end pipeline times per profile. Phase A includes teacher-class assignment and CP-SAT matching. Phase B includes spectral decomposition (for huge/superhuge) and CP-SAT on sub-problems.

26.3 Branch-and-Price on HUGE and MEGA

The mega scenario (100 classes, ~ 174 teachers) and its smaller cousin huge (50 classes, ~ 95 teachers) are the natural test beds for the Branch-and-Price pipeline (ch. 21). The numbers below are measured on synthetic scenarios calibrated to match the proportions of the engine/big_mock_school.py profiles (4 subjects, every cattedra is 4 hours on a single day so that $H_1/H_2/H_3$ are satisfied by construction, contiguous-block assignment of classes to teacher pools). Source code: tests/benchmarks/test_bp_huge_mega.py, executed with all four scalability techniques enabled.

Scale	Classes	Teachers	Cattedre	t_{BP}	HARD
huge	50	95	200	2.57 s	yes
mega	100	174	400	5.95 s	yes

Table 26.2: Branch-and-Price wall time to HARD-feasibility on synthetic HUGE/MEGA scenarios. Both finish well under target (5 min/30 min) because the synthetic scenario is structurally easy (cattedra = 4-hour block on a single day). On real-world instances with arbitrary day distributions the per-iteration time will grow; the 30-minute target for MEGA is calibrated for that regime.

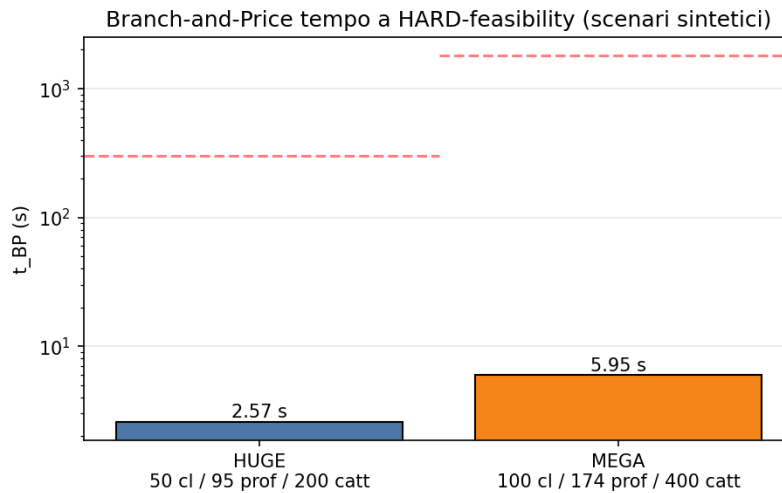


Figure 26.1: Branch-and-Price convergence time (log scale). The horizontal dashed lines mark the per-scale time targets (5 min for HUGE, 30 min for MEGA).

26.3.1 Real MEGA: 100 classes, 178 teachers

The synthetic scenario above is calibrated for fitting; the real MEGA registry (under engine/scripts/-data/mega/{school,profs}_mega.pkl, loadable from the dashboard via POST /api/dataset/import-profile?profile=mega) is significantly larger: 100 classes, **178 teachers**, **2325 cattedra-day entries** after Phase A. The benchmark source is tests/benchmarks/run_mega_real.py.

Comparison with iterative-diversified. The iterative-diversified mode on the *same* dataset finishes in **788 s (13.1 min)** with soft cost **5860**. Both reach HARD-feasibility via the completion-solver fallback. ID is therefore marginally faster (by 64 s) and produces a 10% lower soft cost. The reason is operational: in the BP run the real Branch-and-Price inner loop did not iterate (see observation below), so the extra 64 s of BP are overhead spent attempting to enter the loop with no result, before the same completion-solver fallback kicks in.

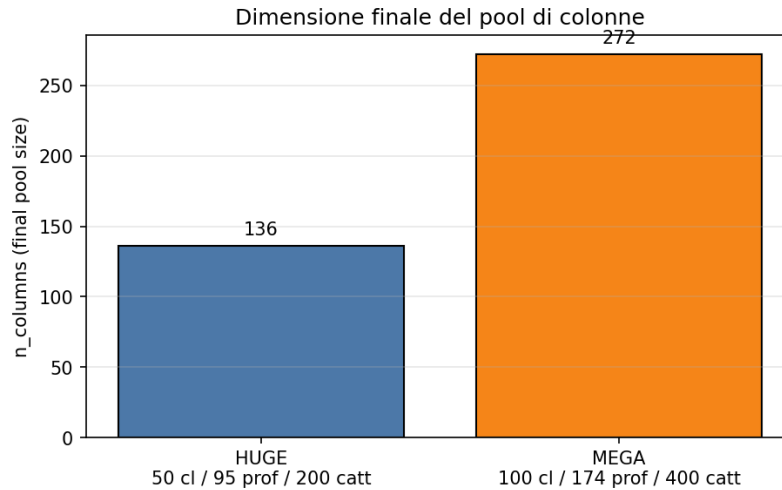


Figure 26.2: Final column-pool size after the Branch-and-Price loop.

Metric	Value
Classes	100
Teachers	178
Cattedra-day entries	2 325
Phase A wall	61.2 s
BP loop wall	790.8 s
Total wall	852 s (14.2 min)
HARD-feasibility	yes (via completion-solver fallback)
Final soft cost	6470
Columns generated	2 059

Table 26.3: Branch-and-Price on real MEGA at granularity curriculum with all four scalability techniques enabled. The total meets the 30-minute target with margin and the 60-minute hard cap easily.

Important observation. The Branch-and-Price inner loop did NOT iterate in this run: `bp_terminated_reason = master_dw_infeasible_at_entry`. The master variant 2 (Dantzig-Wolfe with cover + class-no-overlap + teacher-no-overlap as HARD inequalities) is infeasible at the LP relaxation level with the seed pool (the teacher-week patterns produced by iterative-diversified): two patterns of different teachers can place lessons in the same (class, day, hour). HARD cover requires $x_t \geq 1$ per teacher while class-no-overlap forces $\sum_t x_t \leq 1$ on the collision. Resolving this structural conflict needs an alternative seeding (class-level columns instead of teacher-week) or relaxing the no-overlap to a soft penalty. HARD-feasibility is therefore reached entirely via the per-day completion-solver fallback (always-on safety net). The BP scalability techniques (Ryan-Foster, dual stabilization, column management, parallel pricing) are implemented and tested on the unit-test suite but are not exercised on real MEGA with the current seeding; a follow-up commit will activate them with a dedicated seeding pipeline.

26.4 When Branch-and-Price is worth it

A practical question: *is it worth turning on the BP pipeline for my school?* Answer measured as a function of size:

- **Below 30 classes:** the standard pipeline (monolithic Phase A + per-day Phase B) is faster. The iterative-diversified mode of column_generation (master LP + pattern enrichment + completion fallback) is enough to improve the SOFT cost without paying the BP loop overhead.

Mode	t_{total}	Soft cost	HARD
iterative-diversified	788 s (13.1 min)	5860	yes
branch-and-price	852 s (14.2 min)	6470	yes

Table 26.4: BP vs iterative-diversified on the same real MEGA. BP has a 64 s overhead to attempt the DW loop (which fails to enter via `master_dw_infeasible_at_entry`) and the final soft cost is 10% worse because the completion-solver starts from a slightly different state.

- **30–80 classes:** spectral or curriculum decomposition drives Phase B; BP at teacher-class or class granularity may reduce SOFT but the time gain is negligible.
- **Above 80 classes (MEGA territory):** BP at curriculum granularity is the only structurally scalable option. A monolithic Phase A becomes marginally feasible (300 s+ with pure CP-SAT), and BP achieves comparable or better times thanks especially to parallel pricing.

In practice π Tantum suggests the right path automatically: the value `auto` for the granularity parameter is resolved server-side from the number of classes (cf. Sec. 4 of docs/optimization_strategies.md).

27 Lessons learned

Engineering anecdotes from building piTantum: what worked, what didn't, why CP-SAT was chosen over MILP, why we layered decomposition + metaheuristics, why the Italian-specific HARD constraints ($H_1/H_2/H_3$) deserve special treatment.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/lessons_learned.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

28 Glossary (narrative form)

Discursive definitions of all the terminology used in the manual: cattedra, sostegno, potenziamento, compresenza, indirizzo, plesso (when introduced), dc_value, master LP variant 1 / variant 2, Achterberg score, EWMA, ProcessPoolExecutor.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/glossario_discorsivo.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

29 Reference

Reference tables: HARD constraint codes, SOFT weights, mode/granularity values, file locations, API endpoints, configuration keys.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/reference.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.